Version 1.1

January 2026

DATA STRUCTURES

MODULE 4 - TREE

Author:

MUHAMMAD ILHAM PERDANA. S.Tr.T., M.T.

AHMAD NUR MU'MININ

WAHYU ANDIKA

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS **2**
INTRODUCTION **4**
OBJECTIVES 4
MODULE TARGETS 4
PREPARATION 4
KEYWORDS 4
CONTENTS **5**
1. Fundamental Concepts of Tree Data Structures **5**
1.1. Definition and Characteristics of Trees 5
1.2. Fundamental Terminology in Tree Structures 6
2. Binary Tree **7**
2.1. Definition and Rules of a Binary Tree 7
2.2. Types of Binary Trees 8
2.2.1. On the Basis of Number of Children 8
1. Full Binary Tree 8
2. Skewed Binary Tree 9
2.2.2. On the Basis of Level Completion 9
1. Complete Binary Tree 9
2. Perfect Binary Tree 9
3. Binary Search Tree (BST) **10**
3.1. Fundamental Concepts and Working Principles of BST 10
3.2. BST Analogy 10
4. AVL Tree (Self-Balancing Binary Search Tree) **11**
4.1. Definition and Objectives of an AVL Tree 11
4.2. Balance Concept and Balance Factor 12
4.3. Imbalance Conditions in an AVL Tree 12
5. Traversal in Binary Trees, BSTs, and AVL Trees **12**
5.1. Traversal Concepts and Objectives 12
5.2. Tree Traversal Methods 13
6. Implementation **20**
6.1. Binary Tree 20
6.2. BST 21
6.3. Deletion 22
6.4. Find a Min Value 23
6.5. Find a Max Value 24
6.6. Traversal 24
6.6.1. Inorder 24
6.6.2. Preorder 25
6.6.3. Postorder 26

CODELAB **27**
Mission 1: The Majapahit Empire - Binary Tree 27
DEMO **28**
Task 1 : Station Hierarchy System (General Tree) 28
Task 2: Employee Management System (BST) 29
Grading Rubric 31
Grading Scale 31

INTRODUCTION

OBJECTIVES

1. **Understand the fundamental structure** of a Tree, specifically how a Node class is implemented with data storage and references to left and right children.
2. **Define and identify** key tree terminology including Root, Leaf, Internal Node, Edge, Level, Depth, Height, and Subtree using hierarchical analogies.
3. **Distinguish between different types** of Binary Trees based on their structural properties, such as Full, Skewed, Complete, and Perfect Binary Trees.
4. **Explain the working principle** of a Binary Search Tree (BST), specifically the ordering rule where smaller values are placed to the left and larger values to the right.
5. **Analyze the concept of an AVL Tree** (Self-Balancing BST), focusing on the calculation of the **Balance Factor** and the conditions that trigger rebalancing to maintain efficiency.
6. **Execute Tree Traversal methods**, specifically Preorder, Inorder, and Postorder, to determine the sequence of visiting nodes and processing data.
7. **Build a visual understanding** of hierarchical relationships, such as Ancestors, Descendants, and Siblings, and how they relate to the recursive nature of trees.

MODULE TARGETS

1. **Understand Fundamentals:** Grasp Tree data structures, Node roles, and hierarchy.
2. **Identify Tree Types:** Distinguish between Full, Skewed, Complete, and Perfect Binary Trees.
3. **Implement BST:** Apply Binary Search Tree logic (ordering values).
4. **Analyze AVL Trees:** Understand self-balancing mechanisms and Balance Factors.
5. **Perform Traversals:** Execute Preorder, Inorder, and Postorder methods.
6. **Master Terminology:** Visualize relationships (Root, Leaf, Level, Depth, Height, etc.).

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

   رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop or personal computer
3. Java Development Kit
4. Java Runtime Environment
5. IDE (Intellij IDEA, Eclipse, Netbeans, etc.)
6. Strong motivation and commitment to learning

KEYWORDS

TREE, BST, BINARY TREE

CONTENTS

1. Fundamental Concepts of Tree Data Structures

1.1. Definition and Characteristics of Trees

A tree data structure is a hierarchical structure composed of nodes connected by parent-child relationships. It has a single starting node called the root, which serves as the reference for the entire structure. Unlike arrays or linked lists, a tree does not represent data in a linear order. Each node, except the root, has exactly one parent and may have zero or more children, allowing trees to naturally represent layered or nested data.

Trees are acyclic, meaning there are no closed paths, so each node can be reached through only one unique path from the root. They use levels to indicate the distance of nodes from the root, and the height of a tree is defined by the maximum level. Because of these properties, trees are widely used to model real-world hierarchies such as organizational structures, folder systems, and mathematical expressions, making them a fundamental structure in system and algorithm design.

The following example shows how to build a simple binary tree using the Java programming language. The program defines a Node class to represent a single node in the tree. Each Node object stores one data value and has two references pointing to its left and right children. The relationships between nodes are created manually to show a clear tree structure.

```java
// Node structure
class Node {
    int data;
    Node left, right;
    Node(int d) {
        data = d;
        left = null;
        right = null;
    }
}

class GfG {
    public static void main(String[] args) {
        // Initialize and allocate memory for tree nodes
        Node firstNode = new Node(2);
        Node secondNode = new Node(3);
        Node thirdNode = new Node(4);
        Node fourthNode = new Node(5);

        // Connect binary tree nodes
        firstNode.left = secondNode;
        firstNode.right = thirdNode;
        secondNode.left = fourthNode;

    }
}
```

The Node class acts as the basic building block of the tree. The data variable stores the value of the node, while left and right store references to the left and right child nodes. The constructor initializes the node with a value and sets both child references to null.

In the main method, several nodes are created and connected to form a tree. firstNode serves as the root of the tree. secondNode and thirdNode are assigned as the left and right children of the root. Then, fourthNode is connected as the left child of secondNode. These connections create a simple hierarchical tree structure. This example shows how the theoretical concept of a tree is directly represented in program code.

1.2. Fundamental Terminology in Tree Structures

The tree structure uses a set of standard terms to describe the roles and hierarchical relationships between nodes. Understanding this terminology is important so that readers can correctly read, analyze, and implement tree structures. Each term represents a specific position or function within the structure. To make the concepts easier to understand, each term below is explained using a simple real-life analogy.

A. **Node :** A basic unit that stores data in a tree.
   Analogy: Each individual in the diagram, such as *Grandparent, Parent A, Parent B, Child A1*, and *Child A2*.
B. **Root :** The starting node that becomes the center of the entire structure.
   Analogy: *Grandparent*, who is the oldest ancestor and the top of the family tree.
C. **Child:** A node that is directly below a parent.
   Analogy: *Parent A* and *Parent B* as children of *Grandparent*, and *Child A1* and *Child A2* as children of *Parent A*.
D. **Leaf :** A node that has no children and is located at the end of a branch.
   Analogy: *Child A1* and *Child A2*, who have no descendants.
E. **Internal Node:** A node that has at least one child.
   Analogy: *Grandparent* and *Parent A*, because they have children in the diagram.
F. **Edge:** A connection between two nodes that shows their relationship.
   Analogy: The lines connecting *Grandparent* to *Parent A* and *Parent B*, and *Parent A* to *Child A1* and *Child A2*.
G. **Level :** The position of a node based on its distance from the root.
   Analogy:
   Level 0: *Grandparent*
   Level 1: *Parent A* and *Parent B*
   Level 2: *Child A1* and *Child A2*

H. **Depth :** The number of steps from the root to a specific node.
Analogy: The depth of *Child A2* is two steps: *Grandparent* → *Parent A* → *Child A2*.

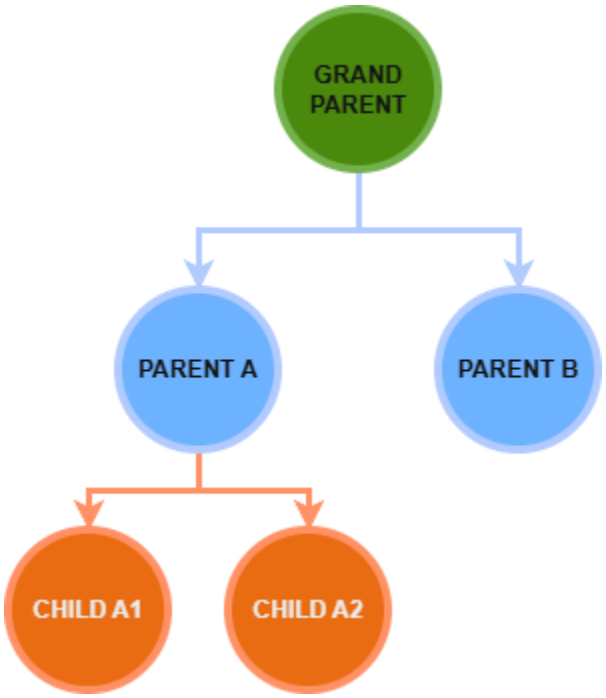
I. **Height :** The longest distance from a node to its deepest leaf.
Analogy: The height of *Parent A* is one step, from *Parent A* to *Child A1* or *Child A2*.

J. **Subtree :** A part of a tree that forms its own tree with a node as a local root.
Analogy: The branch consisting of *Parent A*, *Child A1*, and *Child A2*.

K. **Sibling:** Nodes that share the same parent.
Analogy: *Child A1* and *Child A2*, who both have *Parent A* as their parent.

L. **Ancestor:** All nodes above a given node up to the root.
Analogy: The ancestors of *Child A2* are *Parent A* and *Grandparent*.

M. **Descendant:** All nodes below a given node.
Analogy: The descendants of *Grandparent* are *Parent A, Parent B, Child A1,* and *Child A2*.



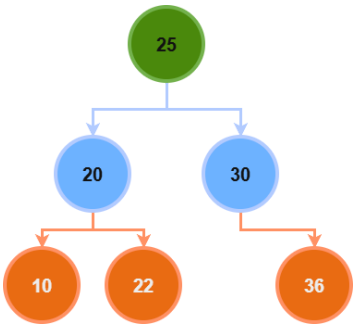
2. Binary Tree

2.1. Definition and Rules of a Binary Tree

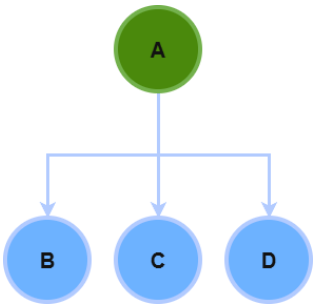
A binary tree is a hierarchical data structure in which each node has at most two children, known as the left child and the right child. This limitation on the number of children distinguishes a binary tree from a general tree and defines its basic structure without considering any ordering of data values. Each node

may have zero, one, or two children, and a node with no children is classified as a leaf. These rules ensure that the structure remains consistent across all levels of the tree.

The positions of the left and right children have distinct structural meanings, so swapping them results in a different binary tree even if the stored values are the same. A binary tree also does not require any ordering relationship between node values, which makes it flexible for various applications and differentiates it from a binary search tree. **Heaps, binary search trees, and AVL trees are examples of data structures that are based on the binary tree model.**



BINARY TREE



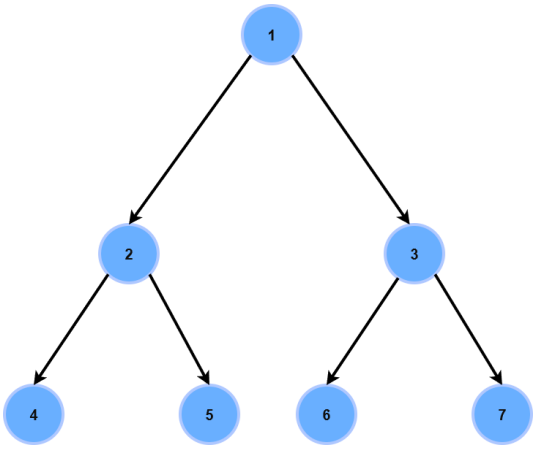
NOT BINARY TREE

2.2. Types of Binary Trees

2.2.1. On the Basis of Number of Children

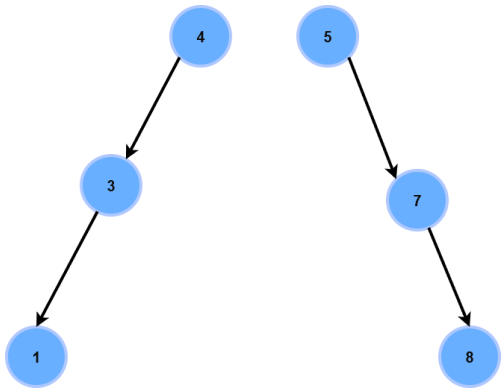
1. Full Binary Tree

A full binary tree is a binary tree in which every node has either two children or no children at all. All internal nodes have exactly two children, while leaf nodes have none. A node with only one child is not allowed in this structure.



2. Skewed Binary Tree

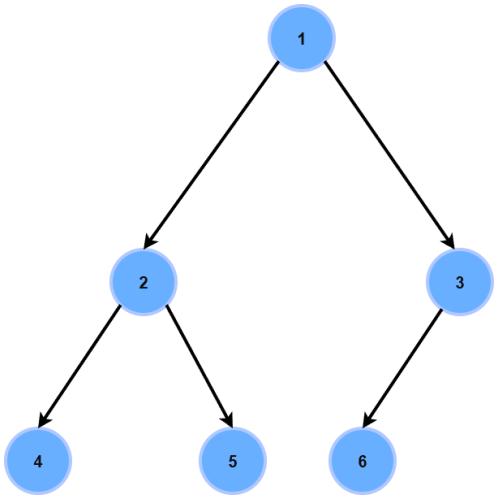
A skewed binary tree is a special case of a degenerate binary tree. In this structure, every node has only one child, which is always on the same side. If the child is always on the left, it is called a left-skewed tree; if always on the right, it is called a right-skewed tree.



2.2.2. On the Basis of Level Completion

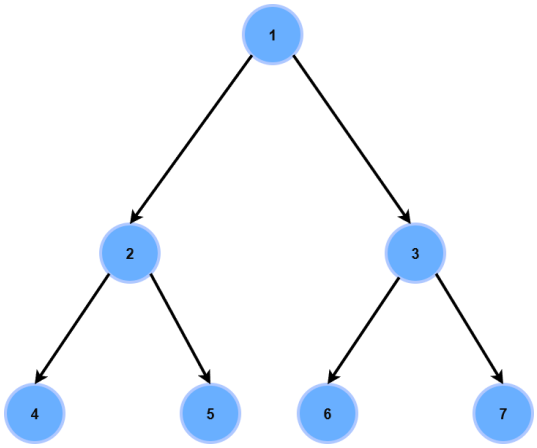
1. Complete Binary Tree

A complete binary tree is a binary tree in which all levels are completely filled except possibly the last level. The last level is filled from left to right without gaps. This structure keeps the tree compact and efficient for storage.



2. Perfect Binary Tree

A perfect binary tree is a binary tree in which all internal nodes have exactly two children and all leaf nodes are at the same level. No level is partially filled, resulting in a fully symmetrical structure.



3. Binary Search Tree (BST)

3.1. Fundamental Concepts and Working Principles of BST

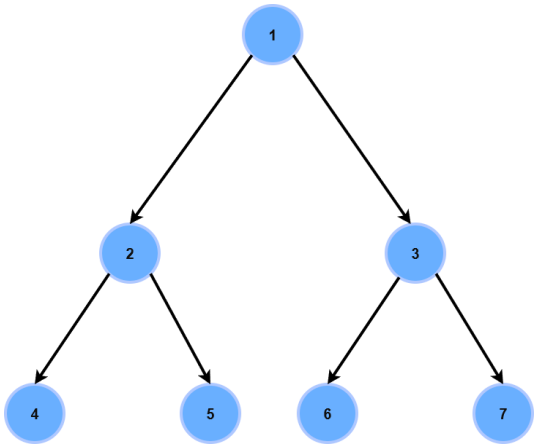
A Binary Search Tree (BST) is a binary tree that follows an ordering rule for its nodes. Each node stores a single key value that determines its position in the tree. Values smaller than the node are placed in the left subtree, while values larger are placed in the right subtree, and this rule is applied repeatedly at every level. This structure supports step-by-step searching based on comparisons. The efficiency of a Binary Search Tree depends on its shape. When the tree is balanced, its height stays relatively small, allowing fast search, insertion, and deletion. When the tree becomes unbalanced, the depth grows and these operations take more time.

3.2. BST Analogy

Imagine a Binary Search Tree (BST) as a train ticket management system based on seat numbers. Each seat is represented by a ticket, and every ticket has a unique seat number that serves as the main reference for data placement.

In this system, all tickets with seat numbers smaller than a reference ticket are placed on the left side, while tickets with larger seat numbers are placed on the right side. This placement rule is applied not only at the top level but also repeatedly within each subgroup of tickets, keeping the entire structure consistent from start to finish.

Each seat number can belong to only one passenger, since two tickets with the same number would cause a placement conflict. When staff review the passenger list from the smallest seat number to the largest, the seats appear in ascending order without any additional sorting. The accuracy and efficiency of finding passenger data depend entirely on following this placement rule, making it the core principle of the Binary Search Tree structure.

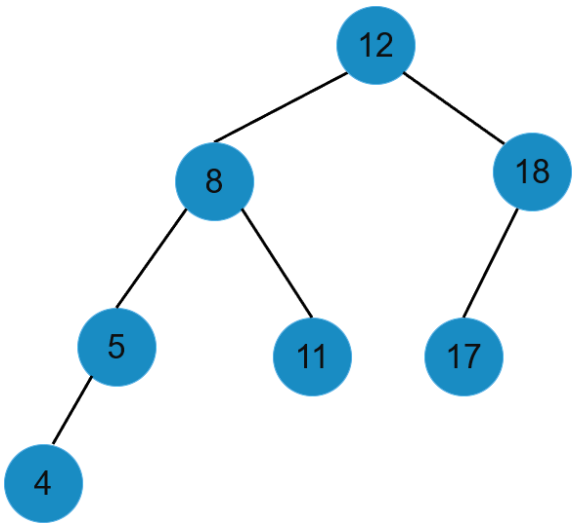


4. AVL Tree (Self-Balancing Binary Search Tree)

4.1. Definition and Objectives of an AVL Tree

An **AVL Tree** is a type of **Binary Search Tree (BST)** that keeps itself balanced automatically. It controls the height difference between the left and right subtrees of each node so the tree does not become too tall on one side. When a node is inserted or removed, the tree adjusts its structure to restore balance.

The main purpose of an AVL Tree is to keep the tree height small, which makes searching, inserting, and deleting fast. It follows the same ordering rules as a normal BST: smaller values go to the left, and larger values go to the right. Even though AVL Trees are more complex and need extra work to maintain balance, they provide stable and efficient performance.



4.2. Balance Concept and Balance Factor

The **balance concept and balance factor in an AVL Tree** focus on keeping the tree structure shallow and stable. Balance is determined by comparing the height of the left subtree and the height of the right subtree at each node. A balanced condition ensures that no branch grows much deeper than the other. This helps keep tree operations consistent and efficient.

The **balance factor** is defined as the difference between the height of the left subtree and the height of the right subtree of a node. It is calculated using the formula **height(left subtree) − height(right subtree)**. This value is checked at every node after structural changes such as insertion or deletion. In an AVL Tree, only balance factor values of −1, 0, or 1 are allowed.

- **0** indicates that the left and right subtrees have the same height.
- **−1 or 1** indicates a height difference of one level, which is still acceptable.
- **Any value outside this range** indicates that the node is unbalanced.

When the balance factor goes beyond the allowed range, a rebalancing process is triggered. This process adjusts the structure to restore balance. The balance factor acts as a key indicator for detecting imbalance. It enables the AVL Tree to correct structural problems in a systematic way.

4.3. Imbalance Conditions in an AVL Tree

Imbalance in an AVL Tree occurs when the height difference between the left subtree and the right subtree exceeds the allowed limit. This condition is detected using the balance factor of a node. In an AVL Tree, a node is considered unbalanced if its balance factor is less than −1 or greater than 1. This strict rule is used to maintain the efficiency of the tree structure.

Imbalance can occur as a result of insertion or deletion operations. An insertion may increase the height of one subtree more than the other, while a deletion may significantly reduce the height of a subtree. These changes affect the nodes along the path from the modified node up to the root. As a result, multiple nodes may need to be checked for balance.

5. Traversal in Binary Trees, BSTs, and AVL Trees

5.1. Traversal Concepts and Objectives

Traversal in a tree structure is defined as a systematic process of visiting every node exactly once using a specific order. This process does not change the tree structure or the stored values. Traversal provides a controlled way to access nodes in a hierarchical structure, which cannot be read directly like linear data

structures. Its primary purpose is to enable complete data access and serve as the foundation for algorithmic interaction with tree data.

Traversal also preserves the order of access required by different applications, since the visiting sequence affects the processing results. Different traversal methods produce different output orders, which are useful for specific tasks such as searching, expression evaluation, and data sorting. Traversal supports both recursive and iterative algorithms and is designed to be efficient by avoiding redundant visits. Therefore, understanding traversal concepts and goals is essential before learning specific traversal techniques.

5.2. Tree Traversal Methods

Tree traversal is a technique used to visit and process every node in a tree in a specific order. The order of visiting nodes determines how the tree data is read, processed, or transformed. The three most common traversal methods are preorder, inorder, and postorder traversal. Each method follows a different visiting sequence and is suited for different purposes.

1. **Preorder traversal**

Preorder traversal follows a visiting order where the parent node is processed first, followed by the left subtree and then the right subtree. The formal order is root, left, right. Each node is visited before any of its children, which means the traversal moves from the top of the tree downward. This rule ensures that the structure of the tree is captured early during traversal and that every node is visited exactly once following this sequence.

Step-by-Step Preorder Traversal
Tree Structure:

- **Root:** A
- **Left Subtree of A:** B (which has children D and E)
- **Right Subtree of A:** C (leaf node)

Step 1 Start at the root **A**.

- **Visit Node A** immediately (Root is processed first).
- Move to the **Left Subtree** (Node B).
- *Current Output:* **A**

Step 2 At Node B.

- **Visit Node B** immediately (Root of this subtree).
- Move to the **Left Subtree** of B (Node D).
- *Current Output:* **A B**

Step 3 At Node D.

- **Visit Node D** immediately.
- D has no left child and no right child.
- *Current Output:* **A B D**

Step 4 Backtrack to the parent of D, which is **Node B**.

- The left subtree of B is done.
- Move to the **Right Subtree** of B (Node E).
- *Current Output:* **A B D**

Step 5 At Node E.

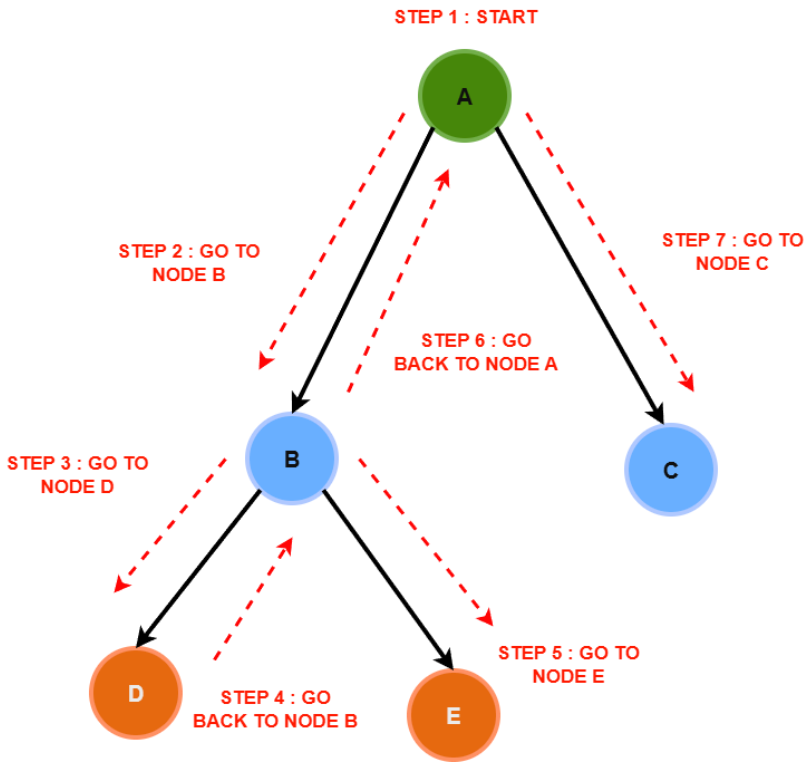
- **Visit Node E** immediately.
- E has no left child and no right child.
- *Current Output:* **A B D E**

Step 6 Backtrack to B (done), then backtrack to **Node A**.

- The entire left subtree of A (B, D, E) is done.
- Move to the **Right Subtree** of A (Node C).
- *Current Output:* **A B D E**

Step 7 At Node C.

- **Visit Node C** immediately.
- C has no left child and no right child.
    - *Current Output:* **A B D E C**



2. **Inorder traversal** visits the left subtree first, then the parent node, and finally the right subtree. This order is defined as *left, root, right*. A node is processed only after its left subtree has been fully visited. When applied to a Binary Search Tree, inorder traversal produces node values in ascending order, making it useful for sorting data, sequential output, and checking whether a BST is correctly structured.

   Step-by-Step Inorder Traversal
   Tree Structure:

   - **Root:** A
   - **Left Subtree of A:** B (which has children D and E)
   - **Right Subtree of A:** C (leaf node)

   Step 1 Start at the root **A**, but **do not** visit it yet.

   - Move to the **Left Subtree** (Node B).
   - *Current Output:* (empty)

Step 2 At Node B, **do not** visit it yet.

- Move to the **Left Subtree** of B (Node D).
- *Current Output:* (empty)

Step 3 At Node D, check for a left child.

- D has no left child.
- **Visit Node D** (Left).
- D has no right child.
- *Current Output:* **D**

Step 4 Backtrack to the parent of D, which is **Node B**.

- The left subtree of B is done.
- **Visit Node B** (Root of this subtree).
- Move to the **Right Subtree** of B (Node E).
- *Current Output:* **D B**

Step 5 At Node E, check for a left child.

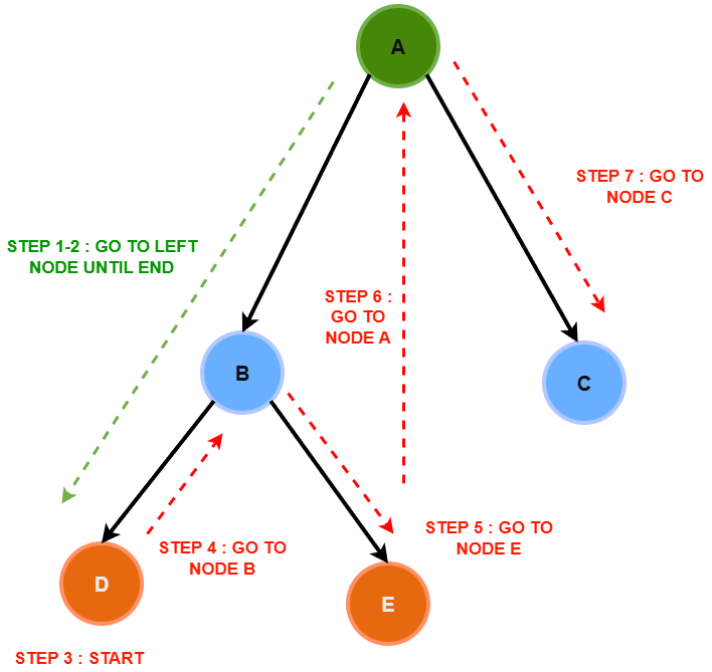
- E has no left child.
- **Visit Node E** (Left/Root logic applies here as it's a leaf).
- E has no right child.
- *Current Output:* **D B E**

Step 6 Backtrack to B (done), then backtrack to **Node A**.

- The entire left subtree of A (B, D, E) is done.
- **Visit Node A** (Root).
- Move to the **Right Subtree** of A (Node C).
- *Current Output:* **D B E A**

Step 7 At Node C, check for a left child.

- C has no left child.
- **Visit Node C** (Left/Root logic applies here).
- C has no right child.
- *Current Output:* **D B E A C**



3. **Postorder traversal** visits the left subtree first, then the right subtree, and processes the parent node last. This order is defined as *left, right, root*. The traversal follows a bottom-up approach, where a node is handled only after all its children are processed. This method is suitable for tasks that depend on subtree results, such as deleting a tree or evaluating postfix expressions.

   Step-by-Step Postorder Traversal
   Tree Structure:

   - **Root:** A
   - **Left Subtree of A:** B (which has children D and E)
   - **Right Subtree of A:** C (leaf node)

   Step 1 Start at the root **A**, but **do not** visit it yet (Root is visited last).

   - Move to the **Left Subtree** (Node B).
   - *Current Output:* (empty)

   Step 2 At Node B, **do not** visit it yet.

   - Move to the **Left Subtree** of B (Node D).
   - *Current Output:* (empty)

Step 3 At Node D, check for children.

- D has no left child.
- D has no right child.
- **Visit Node D** (Left child processed first).
- *Current Output:* **D**

Step 4 Backtrack to the parent of D, which is **Node B**.

- The left subtree of B is done.
- Move to the **Right Subtree** of B (Node E).
- *Current Output:* **D**

Step 5 At Node E, check for children.

- E has no left child.
- E has no right child.
- **Visit Node E** (Right child processed).
- *Current Output:* **D E**

Step 6 Backtrack to the parent of E, which is **Node B**.

- The left child (D) is done.
- The right child (E) is done.
- **Visit Node B** (Root of this subtree processed last).
- *Current Output:* **D E B**

Step 7 Backtrack to the main root, **Node A**.

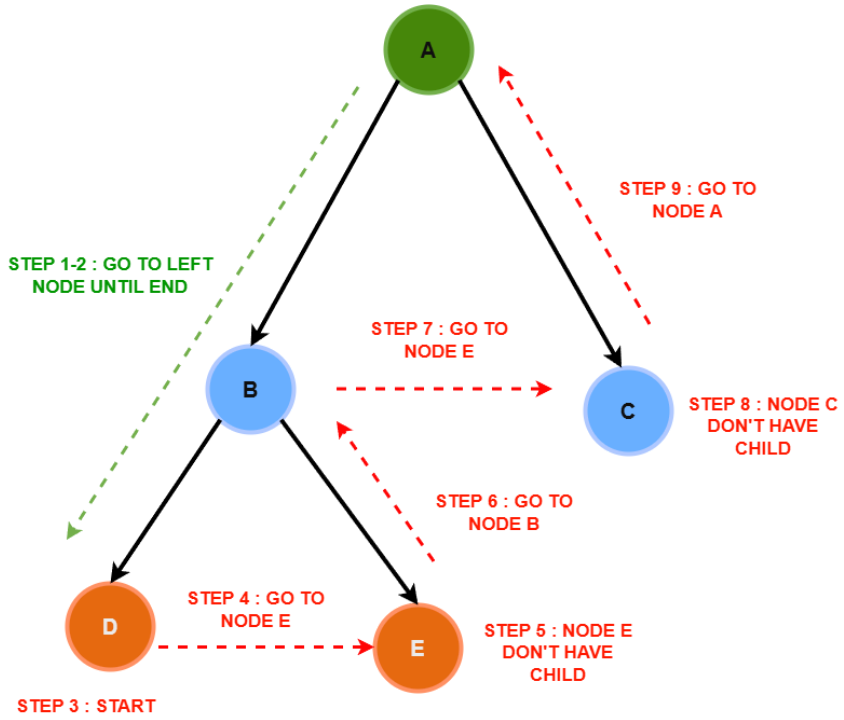
- The entire left subtree of A is done.
- Move to the **Right Subtree** of A (Node C).
- *Current Output:* **D E B**

Step 8 At Node C, check for children.

- C has no left child.
- C has no right child.
- **Visit Node C** (Leaf node logic).
- *Current Output:* **D E B C**

Step 9 Backtrack to the main root, **Node A**.

- The left subtree (B, D, E) is done.
- The right subtree (C) is done.
- **Visit Node A** (Root processed last).
- *Current Output:* **D E B C A**



6. Implementation

6.1. Binary Tree

```java
public class BinaryTree {

    // 1. Node Class
    static class Node {
        int data;
        Node left;
        Node right;

        public Node(int data) {
            this.data = data;
            this.left = null;
            this.right = null;
        }
    }

    Node root;

    public static void main(String[] args) {
        BinaryTree tree = new BinaryTree();

        // Root Node
        tree.root = new Node(1);

        // Children of Root
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);

        // Children of Node 2
        tree.root.left.left = new Node(4);
        tree.root.left.right = new Node(5);

        // Visualization Checking (Manual Print)
        System.out.println("Binary Tree Successfully Created!");
        System.out.println("Root: " + tree.root.data);
        System.out.println("Left Child of Root: " + tree.root.left.data);
        System.out.println("Right Child of Root: " + tree.root.right.data);
        System.out.println("Left Child of Node 2: " + tree.root.left.left.data);
    }
}

```

This Java code demonstrates the manual construction of a Binary Tree, which is a hierarchical data structure where each "Node" acts as a container holding a piece of data and two pointers labeled left and right. The process begins by defining the Node class to store these values and then proceeds in the main method to create the "root" (the top node) with a value of 1. By manually assigning new nodes to the left and right variables of existing ones, the code

builds a specific structure where 1 is the parent of 2 and 3, and 2 is further linked as the parent of 4 and 5. The final section of the code simply prints these values to the console to confirm that each node is correctly connected to its parent, effectively creating a digital "family tree" of numbers.

6.2. BST

```java
public class BinarySearchTree {

    // 1. Node Class
    static class Node {
        int data;
        Node left;
        Node right;

        public Node(int data) {
            this.data = data;
            this.left = null;
            this.right = null;
        }
    }

    Node root;

    // 2. Insert Method (BST Logic)
    // Automatically sorts data: Smaller -> Left, Larger -> Right
    public void insert(int data) {
        root = insertRec(root, data);
    }

    private Node insertRec(Node root, int data) {
        // Base Case: If the spot is empty, create a new Node here
        if (root == null) {
            root = new Node(data);
            return root;
        }

        // Recursive Step: Go Left or Right down the tree
        if (data < root.data) {
            root.left = insertRec(root.left, data);
        } else if (data > root.data) {
            root.right = insertRec(root.right, data);
        }

        return root;
    }

    public static void main(String[] args) {
        BinarySearchTree bst = new BinarySearchTree();

        // 3. Automated Construction using Insert
        // We input data, the Logic places it correctly.
        bst.insert(50); // Root
        bst.insert(30); // Less than 50 -> Left
        bst.insert(70); // Greater than 50 -> Right
        bst.insert(10); // Less than 30 -> Left of Left
        bst.insert(35); // Greater than 30 -> Right of Left

        // Verification (Manual Print)
        System.out.println("BST Structure Verification:");
        System.out.println("Root: " + bst.root.data); // 50
        System.out.println("Left: " + bst.root.left.data); // 30
        System.out.println("Right: " + bst.root.right.data); // 70
        System.out.println("Left-Left: " + bst.root.left.left.data); // 10
        System.out.println("Left-Right: " + bst.root.left.right.data); // 35
    }
```

The provided Java code demonstrates the implementation of a Binary Search Tree (BST), a data structure that automatically organizes data in a hierarchical, sorted manner. The program defines a nested Node class to store an integer value and references to left and right children, starting from a central root. The core functionality is driven by a recursive insert method that compares incoming data against existing nodes: smaller values are directed to the left subtree, while larger values are sent to the right. In the main method, the program constructs a sample tree by inserting values such as 50, 30, and 70, then finishes by printing the results to verify that each node was placed in its mathematically correct position according to BST logic.

6.3. Deletion

```java
// 3. DELETION (Remove Data)
    public void delete(int data) {
        root = deleteRec(root, data);
    }

    private Node deleteRec(Node root, int data) {
        if (root == null)
            return root;

        // Find the Node to delete
        if (data < root.data)
            root.left = deleteRec(root.left, data);
        else if (data > root.data)
            root.right = deleteRec(root.right, data);
        else {
            // Node found!

            // Case 1: Leaf node or has 1 child
            if (root.left == null)
                return root.right;
            if (root.right == null)
                return root.left;

            // Case 2: Has 2 children -> Get smallest value from right subtree (Successor)
            root.data = minValue(root.right);

            // Delete that successor
            root.right = deleteRec(root.right, root.data);
        }
        return root;
    }
```

This Java code explains how to delete a specific value from a Binary Search Tree (BST) using a recursive method called `deleteRec`. The process starts by searching for the target data by comparing it to the current node: if the data is smaller, the program moves left, and if it is larger, it moves right. Once the correct node is found, the code handles three possible scenarios: if the node is a "leaf" with no children or has only one child, it simply removes the node and connects the parent to the remaining child. However, if the node has two children, the

program finds the smallest value in the right subtree to take its place, ensuring the tree's sorted order remains intact, and then deletes that duplicate value from the right side.

6.4. Find a Min Value

```java
// Method to find the minimum value in the tree
public int findMin() {
    return findMinRec(root);
}

// A utility function to find the minimum value in BST
private int findMinRec(Node root) {
    if (root == null)
        throw new IllegalStateException("Tree is empty");

    if (root.left == null)
        return root.key;

    return findMinRec(root.left);
}
```

This code defines a method to find the minimum value in a Binary Search Tree by always moving to the left child. The public `findMin` method starts the process by calling a recursive helper with the root node. The helper method first checks whether the tree is empty and throws an exception if it is. Then it checks whether the current node has no left child and if so returns that node key because in a BST the smallest value is stored in the leftmost node. If a left child exists the method calls itself again on the left subtree until it reaches the leftmost node and returns its value.

6.5. Find a Max Value

```java
// Method to find the maximum value in the tree
public int findMax() {
    return findMaxRec(root);
}

// A utility function to find the maximum value in BST
private int findMaxRec(Node root) {
    if (root == null)
        throw new IllegalStateException("Tree is empty");

    if (root.right == null)
        return root.key;

    return findMaxRec(root.right);
}
```

This code finds the maximum value in a Binary Search Tree by repeatedly moving to the right child. The public `findMax` method starts the process by calling a recursive helper with the root node. The helper method first checks whether the tree is empty and throws an exception if it is. Then it checks whether the current node has no right child and if so returns that node key because in a BST the largest value is stored in the rightmost node. If a right child exists the method continues recursively on the right subtree until it reaches the rightmost node and returns its value.

6.6. Traversal

6.6.1. Inorder

```java
// A. INORDER (Left -> Root -> Right) -> Sorted Result
   public void inorder() {
       System.out.print("InOrder: ");
       inorderRec(root);
       System.out.println();
   }

   private void inorderRec(Node root) {
       if (root != null) {
           inorderRec(root.left);
           System.out.print(root.data + " ");
           inorderRec(root.right);
       }
   }
```

This code performs a preorder traversal of a binary tree where each node is visited in the order root then left subtree then right subtree. The preorder method prints a label and starts the traversal by calling a recursive helper with the root node. The helper method checks whether the current node is not null and if so prints the node data first then recursively visits the left child followed by the right child. This process continues until all nodes are visited producing an output that reflects the preorder sequence.

6.6.2. Preorder

```java
// B. PREORDER (Root -> Left -> Right) -> Initial Structure
    public void preorder() {
        System.out.print("PreOrder: ");
        preorderRec(root);
        System.out.println();
    }

    private void preorderRec(Node root) {
        if (root != null) {
            System.out.print(root.data + " ");
            preorderRec(root.left);
            preorderRec(root.right);
        }
    }
```

This code implements preorder traversal in a binary tree where each node is processed in the order root then left subtree then right subtree. The preorder method prints a label and starts the traversal by calling a recursive helper with the root node. The helper method checks whether the current node is not null and if it is valid it prints the node data first then recursively visits the left child and afterward the right child. This continues until all nodes are visited and the output shows the preorder sequence of the tree.

6.6.3. Postorder

```java
// C. POSTORDER (Left -> Right -> Root) -> Delete from bottom
    public void postorder() {
        System.out.print("PostOrder: ");
        postorderRec(root);
        System.out.println();
    }

    private void postorderRec(Node root) {
        if (root != null) {
            postorderRec(root.left);
            postorderRec(root.right);
            System.out.print(root.data + " ");
        }
    }
```

This code performs postorder traversal on a binary tree where nodes are visited in the order left subtree then right subtree and finally the root. The postorder method prints a label and starts the traversal by calling a recursive helper with the root node. The helper method checks whether the current node is not null and if it is valid it first recursively visits the left child then the right child and only after both subtrees are processed it prints the current node data. This continues until all nodes are visited producing an output in postorder sequence.

CODELAB

Mission 1: The Majapahit Empire - Binary Tree

Objective: Help Mpu Prapanca organize the historical figures of the Majapahit Empire! You must implement a Binary Search Tree (BST) to index the key figures named in the chronicles.

Core Concepts: The program includes a built-in guide on Tree Fundamentals (Root, Node, Leaf, etc.) printed at the start.

Your Tasks:

1. The Royal Node: Complete the Node class constructor.
2. Coronation (Insert): Implement the BST logic to add figures recursively.
    a. Left: Name alphabetically smaller than Root.
    b. Right: Name alphabetically larger than Root.
3. Appraisal (Traversal):
    a. InOrder: Alphabetical sorting (A-Z).
    b. PreOrder: Hierarchical structure.
    c. PostOrder: Historical archives (Bottom-up).
4. Chronicle Search (Search): Implement logic to find if a specific figure exists in the records.
5. Dynasty Limits (Min/Max): Find the first (Alphabetical Start) and last (Alphabetical End) names in the index.
6. Historical Correction (Deletion): Implement the logic to remove a figure from the tree (Handling cases for 0, 1, or 2 children).

Output:

```
>>> KITAB NEGARAKERTAGAMA: MAJAPAHIT DYNASTY <<<
[Theory] Root: Founding Father, Leaves: The Legacy...

[Structure Visualization (BST Alphabetical)]
        Raden Wijaya (Root)
        /           \
   Jayanegara    Tribhuwana
   /        \
Gajah Mada  Kertanegara
    \
   Hayam Wuruk

1. InOrder (Alphabetical Sort):
Gajah Mada -> Hayam Wuruk -> Jayanegara -> Kertanegara -> Raden Wijaya -> Tribhuwana ->

2. PreOrder (Royal Decree Structure):
Raden Wijaya -> Jayanegara -> Gajah Mada -> Hayam Wuruk -> Kertanegara -> Tribhuwana ->

3. PostOrder (Historical Archives):
Hayam Wuruk -> Gajah Mada -> Kertanegara -> Jayanegara -> Tribhuwana -> Raden Wijaya ->

>>> UPDATING CHRONICLES (SEARCH & DELETE) <<<
Is 'Gajah Mada' in the tree? YES
Is 'Lembu Sora' in the tree? NO
First Alphabetical Name: Gajah Mada
Last Alphabetical Name:  Tribhuwana
Removing 'Jayanegara' from history...

1. InOrder (Alphabetical Sort):
Gajah Mada -> Hayam Wuruk -> Kertanegara -> Raden Wijaya -> Tribhuwana ->
```

You can clone the project from this GitHub repository, specifically from the 'modul4' branch
" **git clone -b modul4** https://github.com/dikameo/DATA-STRUCTURE.git "

DEMO

Task 1 : Station Hierarchy System (General Tree)

Objective

Create a hierarchical system to represent the parent-child relationships between railway stations (e.g., a Main Station managing several Sub-stations). A General Tree structure is required because a station can have any number of child sub-stations.

Detailed Requirements

1. Class `StationNode`

Create a node class to represent a single station in the general tree.

- **Attributes:**
  - `stationCode` (String): Unique identifier for the station.
  - `stationName` (String): Display name of the station.
  - `region` (String): The region the station belongs to.
  - `children` (List<StationNode>): A dynamic list to hold multiple child nodes (sub-stations).
  - `parent` (StationNode): A reference to the parent station to facilitate upward traversal.
- **Methods:**
  - **`Constructor`:** Initialize the station details and the children list.
  - `addChild`(StationNode child): Add a sub-station to the current station.
  - `removeChild`(StationNode child): Remove a sub-station from the current station.
  - `isLeaf`(): Return true if the station has no sub-stations.
  - `toString`(): Return a string representation of the station.

2. Class StationHierarchy

Create a manager class to handle the tree structure logic.

- **Data Structure:**
  - private StationNode root;
- **Required Operations:**
  - **Set Root:** Define the central or main station of the network.
  - **Add Station:** Add a new station under a specific parent, identified by the parent's station code.
  - **Find Station:** Implement a **recursive search** to locate a station by its code.

- **Remove Station:** Remove a station by its code. You must handle its children (e.g., decide whether to delete them or reassign them to the grandparent).
- **Traversals:**
  - preOrderTraversal: Traverse Root -> Children (Useful for copying the structure).
  - postOrderTraversal: Traverse Children -> Root (Useful for deletion logic).
  - levelOrderTraversal (BFS): Traverse the tree level by level.
- **Tree Stats:** Calculate and return the total number of stations and the height of the tree.

Task 2: Employee Management System (BST)

Objective

Create a system to manage railway employees. A Binary Search Tree (BST) is the required data structure to keep employees sorted by their ID. This allows for efficient Search, Insertion, and Deletion operations (averaging $O(\log n)$ time complexity).

Detailed Requirements

1. Class `Employee`

Create a data model class to represent an employee.

- **Attributes:**
  - `employeeId` (int): Unique identifier for the employee.
  - `name` (String): Full name of the employee.
  - `department` (String): The department the employee belongs to.
  - *(Add other relevant fields as necessary)*.
- **Implements:**
  - `Comparable`<Employee>: You must implement the compareTo method to allow sorting logic based specifically on the employeeId.

2. Class `BSTNode`

Create a node class for the binary tree structure.

- **Attributes:**
  - `data` (Employee): The employee object stored in this node.
  - `left` (BSTNode): Reference to the child node containing a smaller ID.
  - `right` (BSTNode): Reference to the child node containing a larger ID.

3. Class `EmployeeBST`

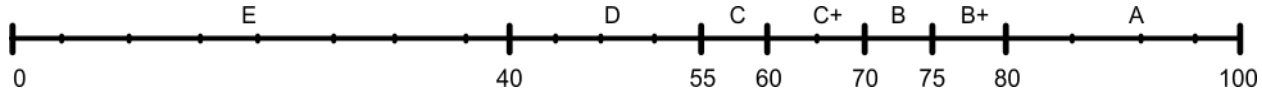
Create a manager class that implements the Binary Search Tree logic.

- **Required Operations:**
  - **Insert:** Add a new employee to the tree while strictly maintaining the BST property (Left Child< Root <Right Child).
  - **Search:** Implement an efficient algorithm to find an employee by their ID.
  - **Delete:** Remove an employee by ID. You must correctly handle the three deletion cases:
    - Node is a **Leaf** (no children).
    - Node has **One Child**.
    - Node has **Two Children** (requires finding the in-order successor or predecessor).
  - **Traversals:**
    - inorderTraversal: Left ->Root-> Right (This must result in a **sorted** list of IDs).
    - preorderTraversal: Root-> Left ->Right.
    - postorderTraversal: Left ->Right -> Root.
  - **Min/Max:** Implement methods to find the employee with the smallest ID (minimum) and the largest ID (maximum).

Grading Rubric

Assessment Aspect	Points
Codelab	**Total 10%**
The program successfully executes all Binary Search Tree operations, including insert, search, delete, and findMin,findMax, without errors	5%
The program successfully executes all Transversal operations, including Inorder, postorder, preorder without errors	5%
Demo	**Total 90%**
Task 1	**45%**
Implement Tree	10%
Running Smoothly	10%
Answer Questions	25%
Task 2	**45%**
Implement BST	10%
Running Smoothly	10%
Answer Questions	25%

Grading Scale



A = (81 - 100) → Sepuh

B+ = (75 - 80) → Very Good

B = (70 - 74) → Good

C+ = (60 - 69) → Fairly Good

C = (55 - 59) → Fair

D = (41 - 54) → Poor

E = (0 - 40) → Bro really...